EAST WEST UNIVERSITY
Department of Computer Science and Engineering
B.Sc. in Computer Science and Engineering Program
Midterm Assessment Project, Summer 2025 Semester

Course:	**CSE407 Green Computing, Section: 01**
Student Name:	Md. Asifuzzaman
Student ID:	2021-3-60-167

Project Title: Dynamic IoT Dashboard for Real-time Device Monitoring and Control.

Checklists:

☑ Real time monitoring of 100 devices with dynamic status indicators.
☑ Device control with the ability to toggle devices ON/OFF.
☑ Dynamic detailed view showing voltage, current and power consumption for each room.
☑ Historic data visualization for power usage with data-range selection.
☑ Real time billing calculation in Bangladeshi Taka (TK)
☑ Data management with database and allow download in CSV file, automatically download multiple files when size limit is reached.
☑ Connectivity hosted on a local server and publicly accessible via the internet.

Declaration:

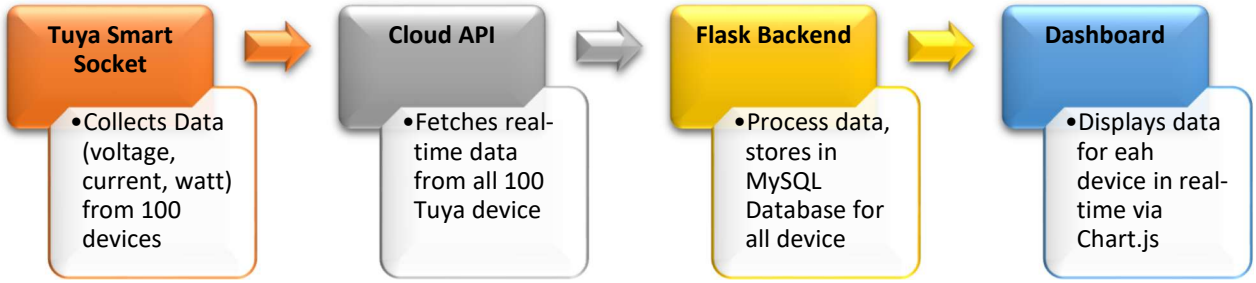
I declare that the project was done by myself and the submitted information is free from any unfair means. I complied with all the relevant legal and organizational requirements.

Signature: _____
Name: Md. Asifuzzaman
Date: 14-08-2025

Executive Summary:

This project focuses on the development of a dynamic, web-based dashboard to monitor and control 100 **Tuya WiFi Smart Socket** in real-time. Each device is represented on the dashboard with a status indicator that shows the device status as **ON** or **OFF**. The system also allows users to **remotely toggle devices** and displays data like **voltage, current, and wattage**. This information appears on a web dashboard created with **Flask** and **Chart.js**. Additionally, the system calculates **real-time billing costs** in **Bangladeshi Taka (TK)**. The recorded data is saved in the **MySQL Database** and evaluated to determine energy consumption in **kilowatt-hours (kWh)**. The data can be filtered by data range and can be downloaded in **CSV files**, with automatic file creation when the file size exceeds **2MB**. The dashboard in hosted on a local server and **publicly accessible through Internet**.

Flowchart of the Work:



Detailed Description of Each Step:

1. **Planning:**
   - The project started with exploring the available energy monitoring options. After looking at the different options, the **Tuya WiFi Smart Socket (CH126 UK Variant)** was chosen; it could be used in conjunction with IoT platforms and is compatible with Python.
   - The project goal was to develop a system that enables **real-time monitoring and control** of all devices, providing insights into energy consumption and operational costs.
2. **Researching:**
   - I studied the **Tuya API** and found that it can provide **real time data** that includes voltage, current and wattage.

- I confirmed the **Tuya WiFi Smart Socket** could meet the power requirements needed for each device, as well that I could access the data from the **cloud**.

3. **Purchase:**
   - I purchased the Tuya WiFi Smart Socket as required.
   - The smart plug was connected to its corresponding rooms to make sure the energy readings were accurate.
4. **Setup:**
   - I setup and connected each Tuya WiFi Smart Socket to the **Tuya app** and recovered the API keys for use.
   - **Flask** was used to set up the backend to pull data from the **Tuya cloud** and show it on the web dashboard.
5. **Data Collection:**
   - I created a Python script to gather data from the Tuya WiFi Smart Socket with a frequency of **8 second intervals**.
   - Measurements were voltage, current and wattage which were recorded in **MySQL Database** and later analyzed.
6. **Data Processing and Analysis:**
   - The data I collected was analyzed with the aim of determining energy consumption in **kilowatt-hours (kWh).**
   - This involved determining the time between data samples, and determining the average wattage in each section of time.
7. **Data Visualization:**
   - The data was visualized on a **real-time web dashboard**, using Chart.js to depict the energy usage. Data can be **sorted** based on **time range**.
   - The dashboard could create time-series graphs for voltage, current, and power.

Challenges and Hiccups:

- **API Authentications Issues:** API authentication with Tuya initially failed due to expired API keys. After regenerating the API keys and updating them in the backend code, the problem was solved.

- **MySQL Database Handling:** Managing large amount of data for 100 devices posed a challenge in terms of performance. The solution involved efficient database queries to fetch real time data.

Demonstration:

- **Link to Energy Dashboard:** https://asifuzzaman.pythonanywhere.com/
- **Pictures:**

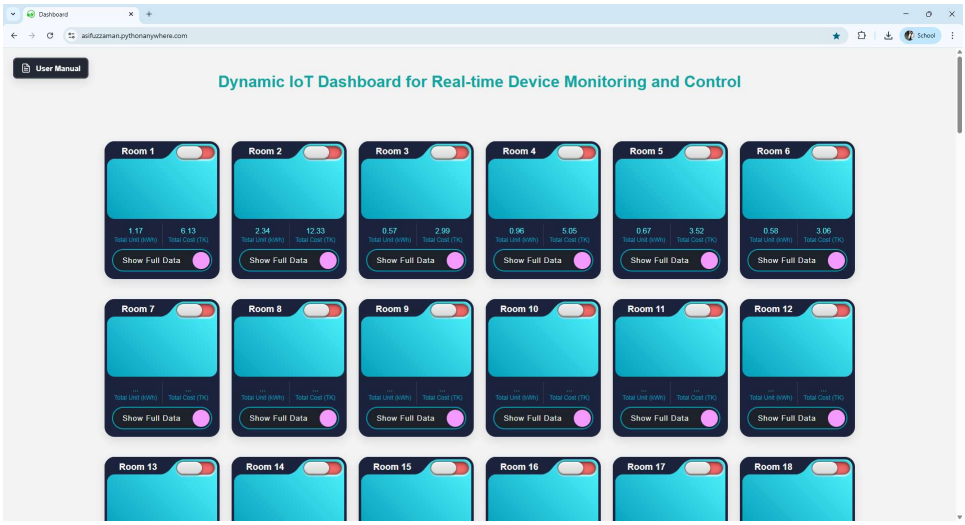


Figure: 01 (Dashboard)

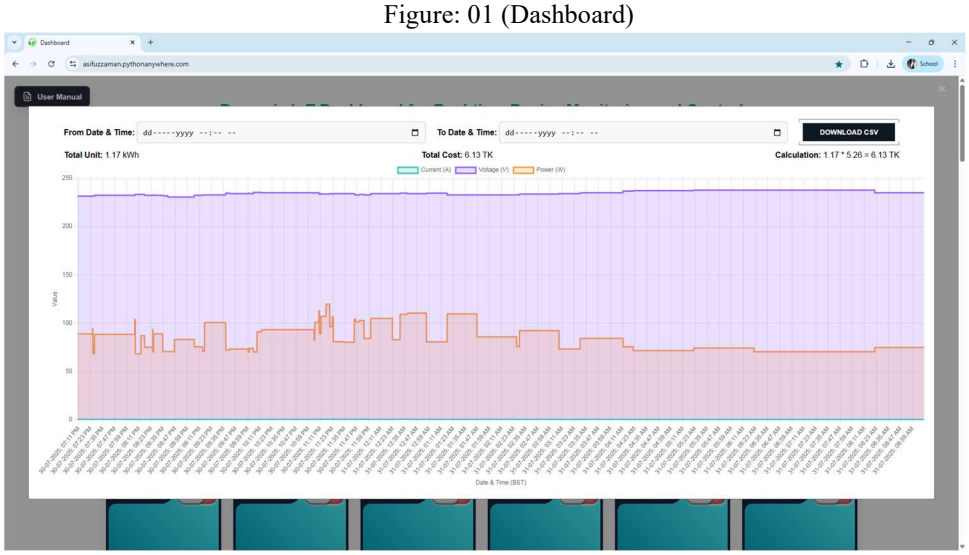


Figure: 02 (Historical Data)

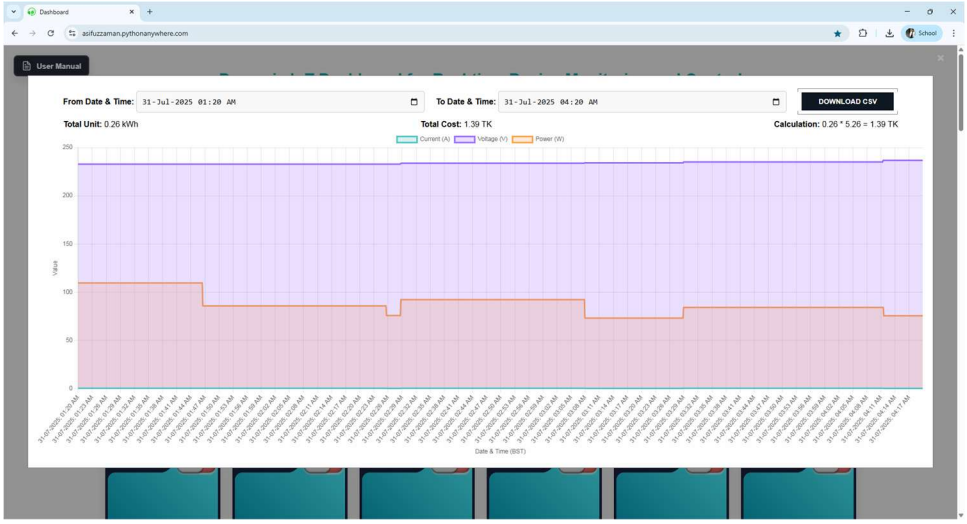
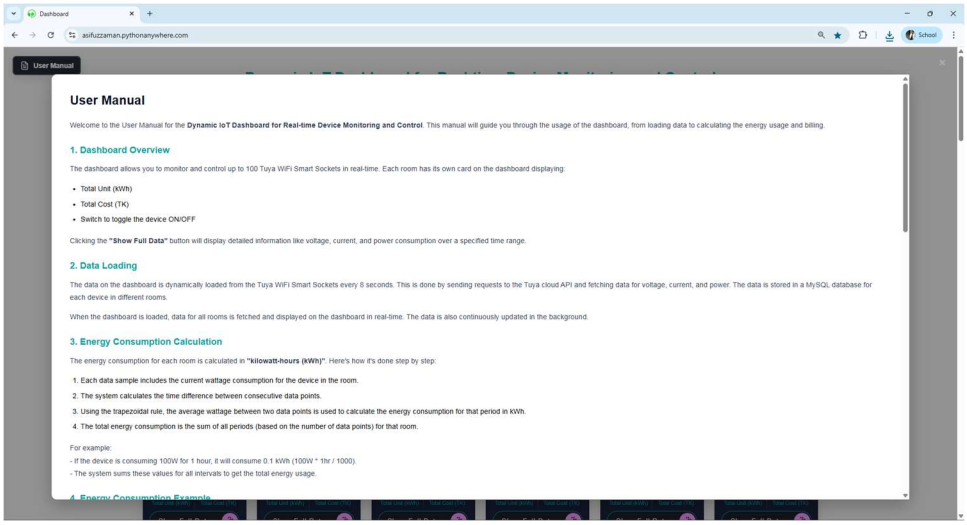


Figure: 03 (Sorted Data)



User Manual

Welcome to the User Manual for the **Dynamic IoT Dashboard for Real-time Device Monitoring and Control**. This manual will guide you through the usage of the dashboard, from loading data to calculating the energy usage and billing.

1. Dashboard Overview

The dashboard allows you to monitor and control up to 100 Tuya WiFi Smart Sockets in real-time. Each room has its own card on the dashboard displaying:

- Total Unit (kWh)
- Total Cost (TK)
- Switch to toggle the device ON/OFF

Clicking the **"Show Full Data"** button will display detailed information like voltage, current, and power consumption over a specified time range.

2. Data Loading

The data on the dashboard is dynamically loaded from the Tuya WiFi Smart Sockets every 8 seconds. This is done by sending requests to the Tuya cloud API and fetching data for voltage, current, and power. The data is stored in a MySQL database for each device in different rooms.

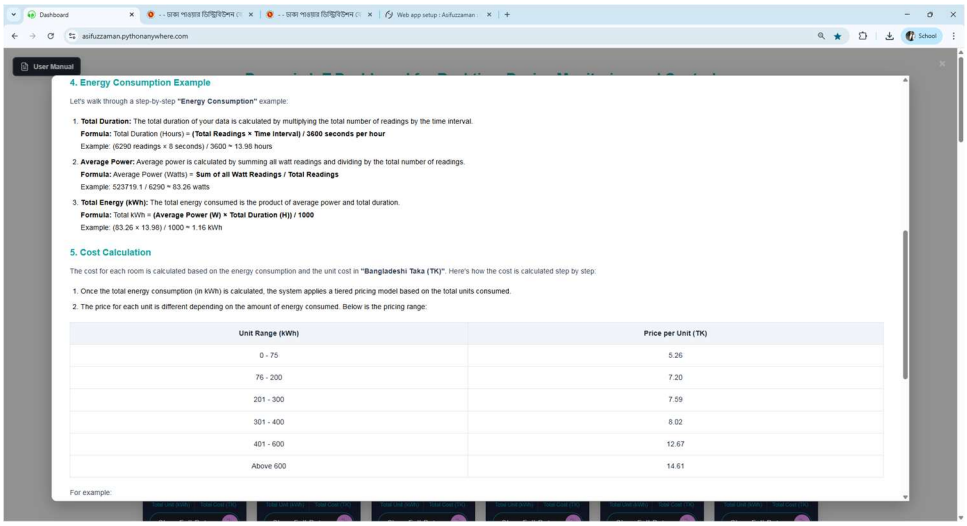
When the dashboard is loaded, data for all rooms is fetched and displayed on the dashboard in real-time. The data is also continuously updated in the background.

3. Energy Consumption Calculation

The energy consumption for each room is calculated in **"kilowatt-hours (kWh)"**. Here's how it's done step by step:

1. Each data sample includes the current wattage consumption for the device in the room.
2. The system calculates the time difference between consecutive data points.
3. Using the trapezoidal rule, the average wattage between two data points is used to calculate the energy consumption for that period in kWh.
4. The total energy consumption is the sum of all periods (based on the number of data points) for that room.

For example:
- If the device is consuming 100W for 1 hour, it will consume 0.1 kWh (100W * 1hr / 1000).
- The system sums these values for all intervals to get the total energy usage.



4. Energy Consumption Example

Let's walk through a step-by-step **"Energy Consumption"** example:

1. **Total Duration:** The total duration of your data is calculated by multiplying the total number of readings by the time interval.
   Formula: Total Duration (Hours) = **(Total Readings × Time interval) / 3600 seconds per hour**
   Example: (6290 readings × 8 seconds) / 3600 ≈ 13.98 hours
2. **Average Power:** Average power is calculated by summing all watt readings and dividing by the total number of readings.
   Formula: Average Power (Watts) = **Sum of all Watt Readings / Total Readings**
   Example: 523719.1 / 6290 ≈ 83.26 watts
3. **Total Energy (kWh):** The total energy consumed is the product of average power and total duration.
   Formula: Total kWh = **(Average Power (W) × Total Duration (H)) / 1000**
   Example: (83.26 × 13.98) / 1000 ≈ 1.16 kWh

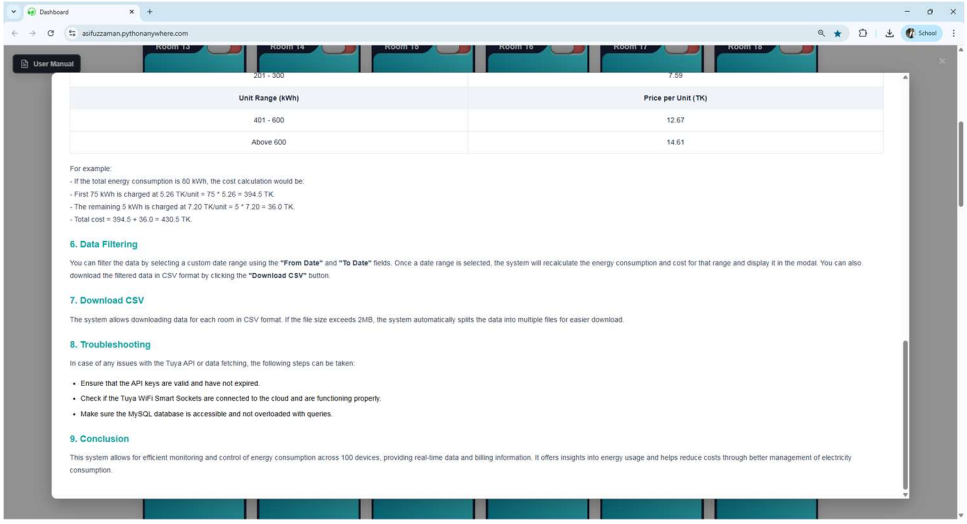
5. Cost Calculation

The cost for each room is calculated based on the energy consumption and the unit cost in **"Bangladeshi Taka (TK)"**. Here's how the cost is calculated step by step:

1. Once the total energy consumption (in kWh) is calculated, the system applies a tiered pricing model based on the total units consumed.
2. The price for each unit is different depending on the amount of energy consumed. Below is the pricing range:

Unit Range (kWh)	Price per Unit (TK)
0 - 75	5.26
76 - 200	7.20
201 - 300	7.59
301 - 400	8.02
401 - 600	12.67
Above 600	14.61

For example:



Unit Range (kWh)	Price per Unit (TK)
401 - 600	12.67
Above 600	14.61

For example:
- If the total energy consumption is 80 kWh, the cost calculation would be:
- First 75 kWh is charged at 5.26 TK/unit = 75 * 5.26 = 394.5 TK.
- The remaining 5 kWh is charged at 7.20 TK/unit = 5 * 7.20 = 36.0 TK.
- Total cost = 394.5 + 36.0 = 430.5 TK.

6. Data Filtering

You can filter the data by selecting a custom date range using the **"From Date"** and **"To Date"** fields. Once a date range is selected, the system will recalculate the energy consumption and cost for that range and display it in the modal. You can also download the filtered data in CSV format by clicking the **"Download CSV"** button.

7. Download CSV

The system allows downloading data for each room in CSV format. If the file size exceeds 2MB, the system automatically splits the data into multiple files for easier download.

8. Troubleshooting

In case of any issues with the Tuya API or data fetching, the following steps can be taken:

- Ensure that the API keys are valid and have not expired.
- Check if the Tuya WiFi Smart Sockets are connected to the cloud and are functioning properly.
- Make sure the MySQL database is accessible and not overloaded with queries.

9. Conclusion

This system allows for efficient monitoring and control of energy consumption across 100 devices, providing real-time data and billing information. It offers insights into energy usage and helps reduce costs through better management of electricity consumption.

Figure: 04 (User Manual)

Discussion on Issues:

Checklist of Issues	Remarks (if any)
*Planning and researching?	Thorough research on the Tuya WiFi Smart Socket and its compatibility with IoT systems
Data Collected?	Data collected every 8 seconds for voltage, current, and wattage.
Data Stored?	Data stored in MySQL database for analysis.
Data Displayed in the dashboard?	Real-time data displayed using Chart.js in a web dashboard.
*Realtime or stored data?	Real-time data collected and processed for 100 devices.
*Wattage of the chosen device?	Correct wattage calculated based on the real-time data.
*Wattage of the measuring equipment?	Tuya WiFi Smart Socket rated for the necessary wattage.
*AC Power: Why/not apparent power? Why/not Instantaneous Power? AC power vs. DC power?	Used real-time wattage (apparent power) instead of instantaneous power. AC power was relevant for the device.

*Documentation	Proper documentation of the setup and code for reproducibility.
*Safety: Electrical Insulation and Isolation?	Ensured the Tuya plug met safety standards for insulation and isolation.
*Caution with overclocking/flushing	No overclocking involved; device settings were kept standard.
*API Issues: Did you get it? How? If not, how did you solve this problem?	API keys obtained successfully after regeneration, solving the authentication issue.
*UI/UX issues? Standard components of an energy dashboard?	Clean, functional dashboard with minimal UI issues.
*User Manual?	Basic guide provided for system setup and dashboard usage.
*Future Extensions and Limitations?	Future extensions could include more than 100 devices monitoring. Limitation - only 100 devices monitored at this time.
*Installation, Operation and Maintenance?	Easy installation; regular monitoring needed for 100 devices. Maintenance required if API keys expire.
*Recurring costs	No recurring costs except electricity consumption.
*Cost Accounting?	Initial setup costs considered (100 Tuya WiFi Smart Socket purchase).
*Business Aspects? Cost savings and ROI? Value of this product/service? Justification?	Potential for energy savings by optimizing energy use; ROI can be seen in long-term electricity bills.
*Reliability? Never failed? Any fail-safe mechanisms?	Reliable in testing, no failures encountered, no automatic fail-safe built into the system.

*Accuracy? Calibration?	Accuracy within device tolerance; no calibration done but consistent readings observed.
*Data quality? Sampling rate? Crosstalk and interference? Accuracy and calibration?	High-quality data with minimal errors. Data sampled every 8 seconds to ensure tracking of energy trends.
*Scalability?	Scalable to monitor multiple devices with slight adjustments in the code.
*Interoperability?	Interoperable with other IoT devices and platforms like Tuya API.
*Data Security? Important or not in this case?	Data security not a primary concern as no personal data is being collected, only energy consumption data.
*Compliant with regulations?	No specific regulations were applicable, but standard safety practices followed.
*Environmental Impacts? PESTLE analysis?	Conducted a PESTLE analysis covering political, economic, social, technological, legal, and environmental aspects.

Appendices:

- **Code:** Energy.py

```
import os
import json
import threading
import time
from datetime import datetime
from flask import Flask, render_template, jsonify, request, Response
from flask_cors import CORS
import tinytuya
import mysql.connector
from io import StringIO
import csv

app = Flask(__name__)
CORS(app)
```

```
# ─── Tuya Cloud Configuration ──────────────────────────────────────────────────────────
API_REGION = "eu"
API_KEY = "jpemtvyqmyvtytvcpywj"
API_SECRET = "bbe0b8e88059472bb0c50e1735537e03"

# ─── Device and Room Mapping ──────────────────────────────────────────────────────────
ROOM_DEVICE_MAP = {
    1: "bf2ea6423fde0e3a5bu0lt", #Only 1 device used for now
}

# ─── Cloud Device Initialization ──────────────────────────────────────────────────────
cloud = tinytuya.Cloud(
    apiRegion=API_REGION,
    apiKey=API_KEY,
    apiSecret=API_SECRET,
    apiDeviceID=next(iter(ROOM_DEVICE_MAP.values())),
)


# ─── MySQL Database Connection Setup ──────────────────────────────────────────────────
def get_db_connection():
    return mysql.connector.connect(
        host="Asifuzzaman.mysql.pythonanywhere-services.com",
        user="Asifuzzaman",
        password="l1v1ng1nh3ll25",
        database="Asifuzzaman$default",
    )


def create_db():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(
            """
            CREATE TABLE IF NOT EXISTS room_data (
```

```
            id INT AUTO_INCREMENT PRIMARY KEY,
            timestamp DATETIME,
            room_id INT,
            connected BOOLEAN,
            power_on BOOLEAN,
            current FLOAT,
            voltage FLOAT FLOAT,
            watt FLOAT
        )
    """
    )
    conn.commit()
except mysql.connector.Error as err:
    print(f"Error creating table: {err}")
finally:
    if "conn" in locals() and conn.is_connected():
        cursor.close()
        conn.close()


# ─── Data Handling ──────────────
```

```
    file_path = os.path.join(DATA_DIR, DATA_FILE_TEMPLATE.format(room_id))
    with open(file_path, "w") as f:
        json.dump(data, f, indent=2)


def insert_data_to_db(room_id, data):
    """Insert data into MySQL database."""
    conn = get_db_connection()
    cursor = conn.cursor()

    for data_point in data:
        cursor.execute(
            """
            INSERT INTO room_data (timestamp, room_id, connected, power_on, current,
voltage, watt)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
        """,
            (
                data_point["timestamp"],
                room_id,
                data_point["connected"],
                data_point["power_on"],
                data_point["current"],
                data_point["voltage"],
                data_point["watt"],
            ),
        )

    conn.commit()
    cursor.close()
    conn.close()


# ─── Data Collection
──────────────
```

```
"""
Thread function to periodically collect data for a single device and save it.
"""
global cloud
print(f"Starting data collection for Room {room_id} with device ID {device_id}")
data_history = load_data(room_id)

while is_collecting.get(room_id, False):
    try:
        result = cloud.getstatus(device_id) if cloud else None
        current_time = datetime.now().isoformat()

        if result and result.get("success", False):
            status_data = result.get("result", [])
            power_on, current, voltage, watt = False, 0, 0, 0
            for item in status_data:
                if item.get("code") == "switch_1":
                    power_on = item.get("value", False)
                elif item.get("code") == "cur_current":
                    current = item.get("value", 0) / 1000
                elif item.get("code") == "cur_voltage":
                    voltage = item.get("value", 0) / 10
                elif item.get("code") == "cur_power":
                    watt = item.get("value", 0) / 10

            data_point = {
                "timestamp": current_time,
                "room_id": room_id,
                "connected": True,
                "power_on": power_on,
                "current": current,
                "voltage": voltage,
                "watt": watt,
            }
        else:
            data_point = {
                "timestamp": current_time,
                "room_id": room_id,
                "connected": False,
                "power_on": False,
```

```
                "current": 0,
                "voltage": 0,
                "watt": 0,
            }

        data_history.append(data_point)
        save_data(room_id, data_history)
        insert_data_to_db(room_id, [data_point])
    except Exception as e:
        print(f"Data collection error for Room {room_id}: {e}")
    time.sleep(7)


# ─── Device Control ──────────────
```

```
def set_device_power(device_id, state):
    """
    Sets the power state of a Tuya device using the a reliable command format.
    """
    try:
        cloud.sendcommand(
            device_id, {"commands": [{"code": "switch_1", "value": state}]}
        )
        return True
    except Exception as e:
        print(f"Error setting device power for {device_id}: {e}")
        return False


# ─── Routes ──────────────
```

```
            return jsonify({"success": True, "power_on": True})
        return jsonify({"success": False, "error": "Failed to set power"}), 500
    return jsonify({"error": "Room not found"}), 404


# Modified route to turn device OFF
@app.route("/off/<int:room_id>")
def power_off(room_id):
    device_id = ROOM_DEVICE_MAP.get(room_id)
    if device_id:
        success = set_device_power(device_id, False)
        if success:
            return jsonify({"success": True, "power_on": False})
        return jsonify({"success": False, "error": "Failed to set power"}), 500
    return jsonify({"error": "Room not found"}), 404


@app.route("/api/status/<int:room_id>")
def get_status_api(room_id):
    device_id = ROOM_DEVICE_MAP.get(room_id)
    if device_id:
        status_data = get_device_status(device_id)
        return jsonify(status_data)
    return jsonify({"error": "Invalid room ID"}), 400


@app.route("/api/toggle/<int:room_id>", methods=["POST"])
def toggle_power_api(room_id):
    device_id = ROOM_DEVICE_MAP.get(room_id)
    if device_id:
        current_status = get_device_status(device_id)
        new_power_state = not current_status.get("switch", False)
        success = set_device_power(device_id, new_power_state)
        if success:
            return jsonify({"success": True, "power_on": new_power_state})
        return jsonify({"success": False, "error": "Failed to toggle power"}), 500
    return jsonify({"error": "Invalid room ID"}), 400


@app.route("/api/data/<int:room_id>")
```

```
def get_data(room_id):
    try:
        from_time = request.args.get("from")
        to_time = request.args.get("to")

        data = get_data_by_room_and_time_range(room_id, from_time, to_time)

        if data:
            data_list = [
                {
                    "timestamp": entry["timestamp"],
                    "room_id": entry["room_id"],
                    "connected": entry["connected"],
                    "power_on": entry["power_on"],
                    "current": entry["current"],
                    "voltage": entry["voltage"],
                    "watt": entry["watt"],
                }
                for entry in data
            ]
            return jsonify(data_list)
        else:
            return jsonify({"error": f"No data found for Room {room_id}"}), 404

    except mysql.connector.Error as err:
        print(f"Error: {err}")
        return jsonify({"error": f"Database error: {err}"}), 500


@app.route("/api/energy/<int:room_id>")
def calculate_energy(room_id):
    data_history = load_data(room_id)
    total_energy = 0
    if len(data_history) > 1:
        for i in range(1, len(data_history)):
            t1 = datetime.fromisoformat(data_history[i - 1]["timestamp"])
            t2 = datetime.fromisoformat(data_history[i]["timestamp"])
            delta_h = (t2 - t1).total_seconds() / 3600
            avg_watt = (data_history[i]["watt"] + data_history[i - 1]["watt"]) / 2
            total_energy += (avg_watt * delta_h) / 1000
```

```
    return jsonify({"energy_kwh": round(total_energy, 4)})


@app.route("/api/download-csv/<int:room_id>")
def download_csv(room_id):
    """
    Endpoint to download data as a CSV file, with optional date range.
    """
    from_date_str = request.args.get("from")
    to_date_str = request.args.get("to")

    from_time = from_date_str if from_date_str else None
    to_time = to_date_str if to_date_str else None

    data = get_data_by_room_and_time_range(room_id, from_time, to_time)

    if not data:
        return (
            jsonify({"error": "No data found for the specified room and time range."}),
            404,
        )

    csv_content = generate_csv(data)

    file_size_mb = len(csv_content.encode("utf-8")) / (1024 * 1024)

    if file_size_mb <= 2:
        return Response(
            csv_content,
            mimetype="text/csv",
            headers={
                "Content-disposition": f"attachment; filename=room_{room_id}_data.csv"
            },
        )
    else:
        mid_point = len(data) // 2
        data_part1 = data[:mid_point]
        data_part2 = data[mid_point:]

        csv_content1 = generate_csv(data_part1)
```

```
    csv_content2 = generate_csv(data_part2)

    return jsonify(
        {
            "success": True,
            "room_id": room_id,
            "part1_data": csv_content1,
            "part2_data": csv_content2,
            "message": "Data split into two files due to size limit.",
        }
    )


def get_data_by_room_and_time_range(room_id, from_time=None, to_time=None):
    """
    Fetches data from the database for a specific room within an optional
    time range.
    """
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)

    query = "SELECT timestamp, room_id, connected, power_on, current, voltage, watt
FROM room_data WHERE room_id = %s"
    params = [room_id]

    if from_time and to_time:
        query += " AND timestamp BETWEEN %s AND %s"
        params.append(from_time)
        params.append(to_time)

    query += " ORDER BY timestamp"

    cursor.execute(query, tuple(params))
    data = cursor.fetchall()

    cursor.close()
    conn.close()
    return data
```

```
def generate_csv(data):
    """Generates CSV content from a list of dictionaries."""
    if not data:
        return ""

    fieldnames = [
        "timestamp",
        "room_id",
        "connected",
        "power_on",
        "current",
        "voltage",
        "watt",
    ]
    csv_buffer = StringIO()
    writer = csv.DictWriter(csv_buffer, fieldnames=fieldnames)

    writer.writeheader()
    writer.writerows(data)

    return csv_buffer.getvalue()


if __name__ == "__main__":
    create_db()
    if ROOM_DEVICE_MAP:
        first_device_id = list(ROOM_DEVICE_MAP.values())[0]
        cloud = tinytuya.Cloud(
            apiRegion=API_REGION,
            apiKey=API_KEY,
            apiSecret=API_SECRET,
            apiDeviceID=first_device_id,
        )

    for room_id in range(1, 101):
        file_path = os.path.join(DATA_DIR, DATA_FILE_TEMPLATE.format(room_id))
        if os.path.exists(file_path):
            data = load_data(room_id)
            if data:
                insert_data_to_db(room_id, data)
```

```
            print(f"Data for Room {room_id} inserted into the database.")
        else:
            print(f"No data found in {file_path}")

    for room_id, device_id in ROOM_DEVICE_MAP.items():
        is_collecting[room_id] = True
        thread = threading.Thread(target=collect_data, args=(room_id, device_id))
        thread.daemon = True
        thread.start()
        print(f"Data collection thread started for Room {room_id}")

    app.run(debug=True, host="0.0.0.0", port=5005)
```

- **Database:**

```
CREATE TABLE IF NOT EXISTS room_data (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp TEXT,
    room_id INTEGER,
    connected BOOLEAN,
    power_on BOOLEAN,
    current REAL,
    voltage REAL,
    watt REAL
)
```

- **Unit Calculation:**

i) Calculate total Duration:

$$\text{Total Duration (Hours)} = \frac{(\text{Total Reading } \times \text{Time Interval})}{3600 \text{ second per hour}} = \frac{6290 \ \times 8}{3600} \approx 13.98 \text{ Hours}$$

ii) Calculate the Average Power:

$$\text{Average Power (Watts)} = \frac{\text{Sum of all Watt Readings}}{\text{Total Readings}} = \frac{523719.1}{6290} \approx 83.26\ Watts$$

iii) Calculate Total Energy (kWh):

$$\text{Total kWh} = \frac{\text{Average Power (W)} \times \text{Total Duration (H)}}{1000} = \frac{83.26 \ \times 13.98}{1000} \approx 1.16 kWh$$

- **Cost Calculation:** https://dpdc.portal.gov.bd/site/page/27e77af7-1187-436c-9f80-b579d3493a46/-

Unit Range (kWh)	Price per Unit (TK)
0 – 75	5.26
76 – 200	7.20
201 – 300	7.59
301 – 400	8.02
401 – 600	12.67
Above 600	14.61

PESTEL Analysis:

- **Political:** Government regulations around energy consumption and the potential for policy incentives on energy-efficient technology.
- **Economic:** Cost savings on electricity bills due to optimized energy use, potential market for energy monitoring solutions.
- **Social:** How society benefits from better energy management, increasing awareness about energy efficiency.
- **Technological:** The role of IoT in enabling energy management, reliability of the technology, and future improvements.
- **Legal:** Compliance with data protection laws regarding data collection.
- **Environmental:** Positive environmental impact due to reduced energy consumption and carbon footprint.

Conclusion:

This project successfully used **IoT technology** with **Tuya WiFi Smart Sockets** to monitor and control **100 devices**, collecting real-time data on **voltage**, **current**, and **wattage**. By storing data in a **MySQL database** and displaying it through a **dynamic web dashboard**, the system efficiently handled large volumes of data and provided valuable insights into **energy consumption**. The project demonstrated scalability, allowing for future expansion and integration with additional devices or smart home platforms. It also showed potential for **cost savings** through real-time billing and **improved energy efficiency**. With its focus on reducing energy use and promoting **sustainability**, the system sets the stage for future advancements in **energy management technologies** and **smart grid solutions**.